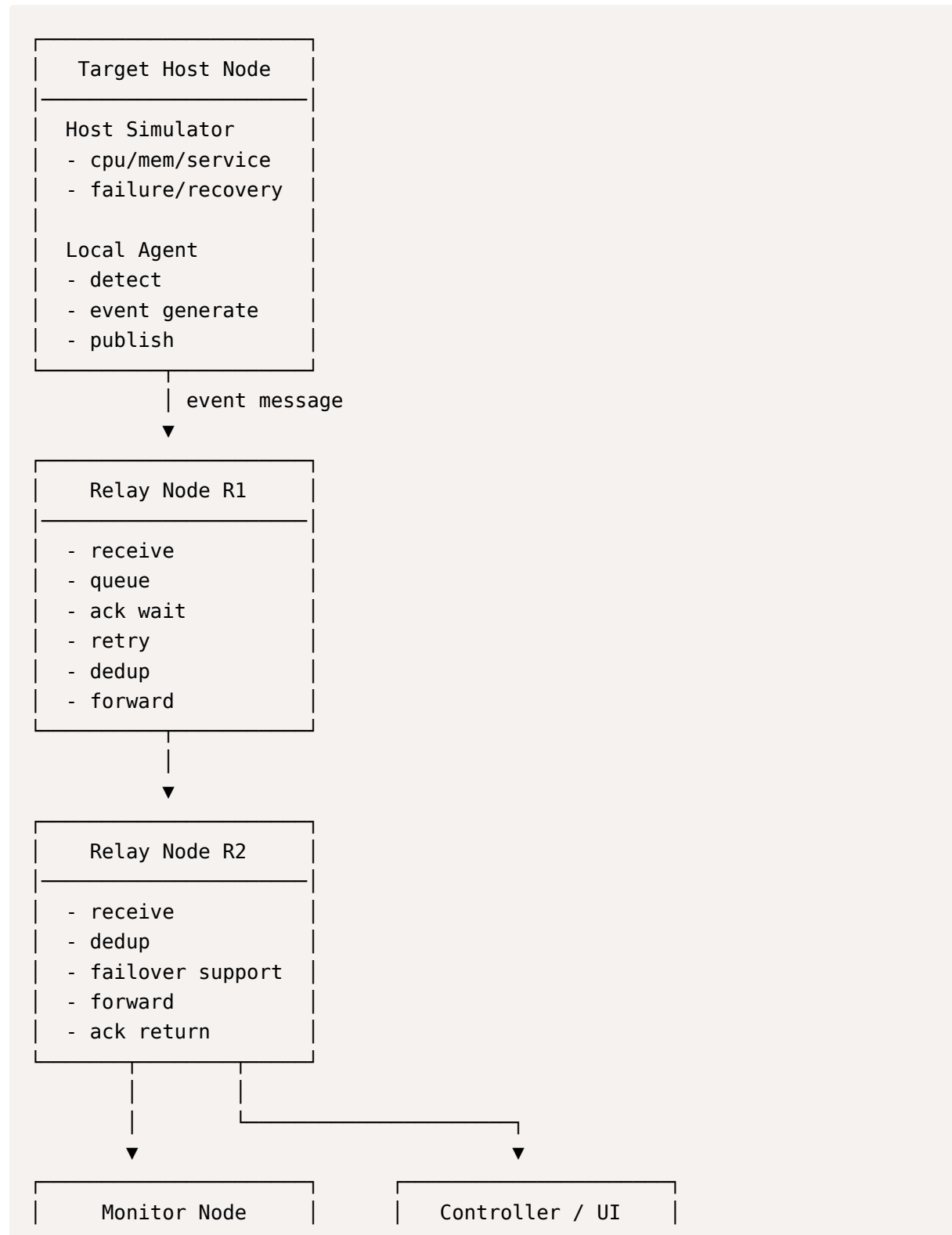
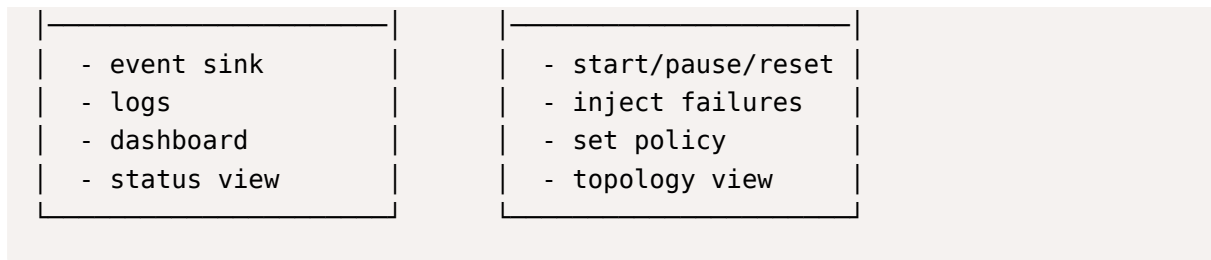


네트워크 프로젝트 아키텍처

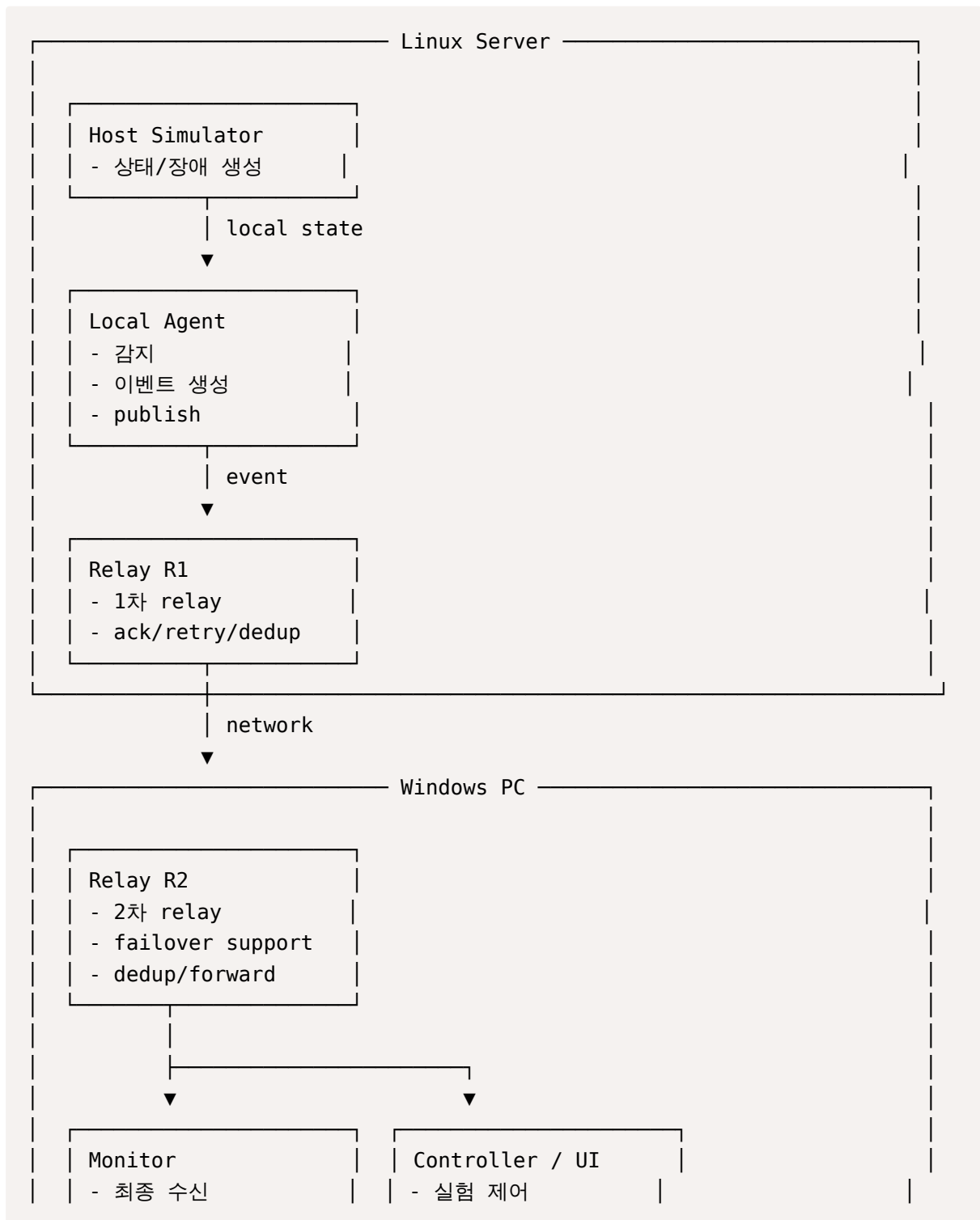
아키텍처

1. 논리 아키텍처





2. 물리적 아키텍처

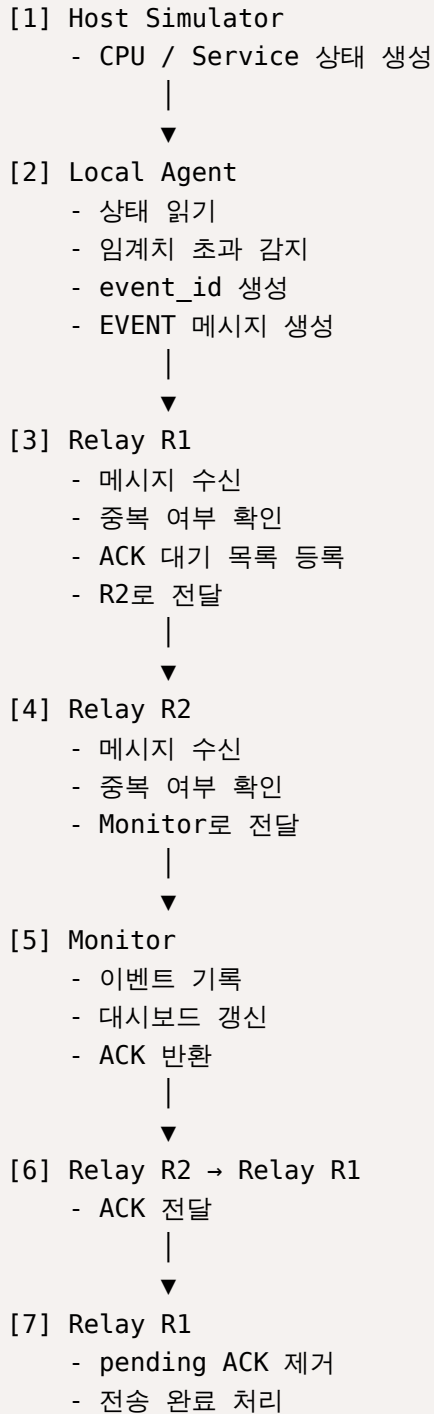


- 로그/시각화

- 장애 주입

플로우차트

1. 정상동작 플로우차트



2. 장애 플로우차트

2-1. Host 장애 발생 플로우차트

- 예: CPU spike, service down



핵심 : 호스트 장애를 시스템이 직접 복구하는 게 아니라, 그 사실을 신뢰성 있게 전달하는 것

2-2. ACK 유실 / 지연 발생 플로우차트



중복 처리 생략(dedup)

↓

ACK만 다시 전송

retry + dedup이 왜 필요한지 보여준다.

2-3. R1 장애 시 플로우차트

Local Agent

↓

R1로 전송 시도

↓

ACK 없음 / 응답 없음

↓

timeout

↓

retry

↓

계속 실패

↓

failover 정책 확인

↓

R2 또는 대체 경로로 전송

↓

Monitor 도달

기본 경로 실패 → timeout → retry → backup route 전환

2-4. R2 장애 시 플로우차트

Local Agent

↓

Relay R1

↓

Relay R2 전송 시도

↓

R2 down

↓

ACK 없음

↓

R1 timeout

↓

retry 반복

↓

정책에 따라

└─ 실패 처리

└─ queue에 보관

└─ direct monitor 경로 시도

Controller/UI 중심 제어

Controller / UI

- └ Start
- └ Pause
- └ Reset
- └ Inject CPU Spike
- └ Inject Service Down
- └ Inject R1 Down
- └ Inject R2 Down
- └ Set Retry Count
- └ Set Timeout
- └ Dedup ON/OFF
- └ Delivery Mode 변경

Controller / UI

└───────────────────▶	Host Simulator	(장애 생성)
└───────────────────▶	Local Agent	(실행 제어)
└───────────────────▶	Relay R1	(retry/timeout/dedup 정책)
└───────────────────▶	Relay R2	(failover/forward 정책)
└───────────────────▶	Monitor	(reset/log clear)

[간단 요약]

- **Host Simulator**가 가상 호스트 상태를 만든다.
- **Local Agent**가 이를 감지해 이벤트 메시지로 바꾼다.
- **Relay R1, R2**가 메시지를 전달하면서 retry, dedup, failover를 수행한다.
- **Monitor**가 최종 수신 후 로그와 상태를 표시한다.
- **Controller/UI**는 전체 시스템의 실행 제어와 장애 주입, 정책 변경을 담당한다.